# CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES

# Software Project Management Plan

*for*

# AudioScholar: Transforming Audio into Actionable Insights for Learners

# AudioScholar: Transforming Audio into Actionable Insights for Learners

**Proponent(s):**

Biacolo, Math Lee L.
Terence, John Duterte
Orlanes, John Nathan
Barrientos, Claive Justin
Alpez, Christian Brent

**Adviser:**

Jasmine A. Tulin

_____

**Consultation Schedule:**

TULIWED11001200

**Date of Submission:**

September 20, 2025

# Table of Contents

Table of Contents ..... 3
1. OVERVIEW ..... 5
1.1 PROJECT SUMMARY ..... 5
1.1.1. Purpose, scope and objectives ..... 5
1.1.2. Assumptions and constraints ..... 6
1.1.3. Project deliverables ..... 7
1.1.4. Schedule and budget summary ..... 7
1.2 EVOLUTION OF PLAN ..... 7
2. REFERENCES ..... 8
3. DEFINITIONS ..... 11
4. PROJECT ORGANIZATION ..... 15
4.1. EXTERNAL STRUCTURE ..... 15
4.2. INTERNAL STRUCTURE ..... 15
4.3. ROLES AND RESPONSIBILITIES ..... 15
5. MANAGERIAL PROCESS PLANS ..... 16
5.1. START-UP PLAN ..... 16
5.1.1. Estimation plan ..... 16
5.1.2. Staffing plan ..... 16
5.1.3. Resource acquisition plan ..... 16
5.1.4. Project staff training plan ..... 16
5.2. WORK PLAN ..... 16
5.2.1. Work activities ..... 16
5.2.2. Schedule allocation ..... 17
5.2.3. Resource allocation ..... 17
5.2.4. Budget allocation ..... 17
5.3. CONTROL PLAN ..... 18
5.3.1. Requirements control plan ..... 18
5.3.2. Schedule control plan ..... 18
5.3.3. Budget control plan ..... 18
5.3.4. Quality control plan ..... 18
5.3.5. Reporting plan ..... 18
5.3.6. Metrics collection plan ..... 18
5.3.7 Risk management plan ..... 19
5.3.8 Project closeout plan ..... 19

6. TECHNICAL PROCESS PLANS ..... 20
6.1 PROCESS MODEL ..... 20
6.2 METHODS, TOOLS, AND TECHNIQUES ..... 20
6.3 INFRASTRUCTURE PLAN ..... 20
6.4 PRODUCT ACCEPTANCE PLAN ..... 21
7. SUPPORTING PROCESS PLANS ..... 22
7.1. CONFIGURATION MANAGEMENT PLAN ..... 22
7.2. VERIFICATION AND VALIDATION PLAN ..... 22
7.3. DOCUMENTATION PLAN ..... 22
7.4. QUALITY ASSURANCE PLAN ..... 22
7.5. REVIEWS AND AUDITS ..... 23
7.6. PROBLEM RESOLUTION PLAN ..... 23
7.7. SUBCONTRACTOR MANAGEMENT PLAN ..... 23
7.8. PROCESS IMPROVEMENT PLAN ..... 23
8. ADDITIONAL PLANS ..... 24
9. PLAN ANNEXES ..... 25
10. INDEX ..... 26

# 1. OVERVIEW

## *1.1 PROJECT SUMMARY*

### *1.1.1. Purpose, scope and objectives*

- **Purpose:** AudioScholar is conceived as an intelligent, multi-user platform designed to record lecture audio and leverage AI-driven summarization techniques to produce structured, actionable insights specifically tailored for learners. It aims to transform the inefficient process of traditional note-taking during lectures into an efficient one, allowing students to focus on active listening and receive well-organized study materials afterwards. The project addresses the inefficiency of traditional note-taking by providing a demonstrably more efficient alternative.

- **Scope:** The AudioScholar software product will be delivered as a dual-platform solution: a mobile application for Android devices (Version 1.0) and a web interface accessible through standard web browsers. The system is designed to capture, summarize, and recommend learning materials based on audio recordings of lectures, not video.

    - **In Scope:** Lecture Recording (Mobile), Audio Upload (Mobile & Web), AI-Powered Summarization (using Google Gemini AI API), Personalized Learning Material Recommendations (initially YouTube), User Authentication and Account Management (Mobile: Email/Password, Google OAuth 2.0, GitHub OAuth 2.0), Optional Cloud Synchronization (Nhost Storage, Firebase Firestore), Optional PowerPoint Integration for Enhanced Summarization (Mobile & Web), Multi-User Support with Data Privacy, Web Interface for Comprehensive Access (Viewing summaries/recommendations, Audio Upload), Freemium Model Implementation.

    - **Out of Scope (Initial Version):** Real-time Transcription, iOS Mobile Platform Support, Web Interface Audio Recording, Multi-language Support (beyond English initially), Recommendation Engine Beyond YouTube (for Free Users).

- **Objectives (SMART Goals):**

    - **Enable Efficient Lecture Recording and Upload:** Develop mobile (Android) recording (offline capable) and upload; web upload. (Target Completion: End of Week 2)

    - **Automate Audio Processing and Generate Structured Lecture Summaries:** Implement server-side AI (Google Gemini) processing for summaries. (Target Completion: End of Week 4)

    - **Provide Personalized Learning Material Recommendations:** Develop server-side analysis and recommendation generation (initially YouTube). (Target Completion: End of Week 6)

    - **Ensure Secure User Authentication and Account Management:** Implement robust mobile authentication (Google/GitHub OAuth, email/password) via Firebase. (Target Completion: End of Week 2)

    - **Enable Optional Cloud Synchronization of User Data:** Provide optional cloud sync (Nhost Storage, Firebase Firestore). (Target Completion: End of Week 8)

- **Integrate PowerPoint Upload for Enhanced Summarization Context:** Allow optional PPT upload via mobile app and web application. (Target Completion: End of Week 6)
- **Develop a Web Interface for Content Access and Management:** Create web interface for access/review/upload. (Target Completion: progressively Weeks 3-8)
- **Implement a Freemium Model for Feature Access Control:** Control feature access based on user status (unauthenticated, free, premium). (Target Completion: End of Week 8)

### *1.1.2. Assumptions and constraints*

- **Assumptions:**
  - Availability of Google Gemini AI API throughout development and operation.
  - Availability and reliability of Firebase services (Firestore, Authentication) and Nhost Storage services.
  - Internet connectivity is required for cloud sync, AI processing, and web interface access (but mobile recording works offline).
  - Target users have access to Android mobile devices and computers with web browsers.
  - User acceptance of the freemium model's tiered access.
  - The Google Gemini AI API will provide reasonably accurate and useful summaries.
- **Constraints:**
  - **Development Timeline:** Project completion within an 8-week academic semester timeframe. The estimated completion date for all objectives and deliverables is May 06, 2025.
  - **Resource Limitations:** Development resources limited to the student team and available open-source tools/cloud services (Firebase free tier, Nhost Storage). AI API usage is managed by staying within free tier limits.
  - **Platform Focus:** Initial mobile development restricted to Android (Kotlin). iOS is out of scope for v1.0.
  - **AI API Dependency:** Core functionality relies on external Google Gemini AI API; changes in availability, pricing, or functionality could impact the project.
  - **Language Constraint:** Initially supports English language lectures only.
  - **Freemium Model Constraints:** Features like background recording disabled for free logged-in users; recommendation sources limited for free users.
  - **No Real-time Transcription:** This feature is not planned for this version.
  - **Web Interface Limitations:** Web interface will not support direct audio recording.

- **UI Paradigm:** Mobile navigation implemented using Navigation Drawer.

### *1.1.3. Project deliverables*

- **Software:**
  - An Android mobile application (Version 1.0, Minimum OS: Android 7.0 Nougat) with features defined in the functional requirements.
  - A web interface allowing users to access recordings, summaries, recommendations, and upload audio files.
  - Server-side application (Spring Boot) for audio processing, summarization, recommendation generation, user management, and cloud storage interaction.
- **Documentation:**
  - Software Requirements Specification (SRS).
  - Software Design Description (SDD).
  - Software Project Management Plan (SPMP).
  - Software Test Document.
  - Final Paper summarizing the project in ACM format.
- **Other:**
  - Usability testing results and user feedback analysis.

### *1.1.4. Schedule and budget summary*

- **Schedule:** The project development timeframe is constrained to 8 weeks, targeting completion by May 06, 2025. Development follows an Agile/Scrum methodology with weekly checkpoints and key increment deadlines (Weeks 2, 4, 6, 8). A detailed project schedule, including task lists, project board, and timeline, is maintained using ClickUp.
- **Budget:** No allocated monetary budget exists currently. The project utilizes free tiers of cloud services (Firebase, Nhost, Google Gemini AI, Render). A future potential allocation of P500 is considered for compensating application testers.

## *1.2 EVOLUTION OF PLAN*

This SPMP document will be maintained and updated as needed throughout the project lifecycle. Changes will be discussed within the team, proposed and implemented by the Team Leader, and tracked via document version control. Significant changes may require consultation with the Project Adviser.

# 2. REFERENCES

[1] Cebu Institute of Technology University, "Software Project Proposal for AudioScholar: Transforming Audio into Actionable Insights for Learners," Cebu City, Philippines, Feb. 22, 2025.

[2] Cebu Institute of Technology University, "Software Requirements Specifications for AudioScholar: Transforming Audio into Actionable Insights for Learners," Version 1.7, Cebu City, Philippines, April 30, 2025.

[3] Cebu Institute of Technology University, "Software Design Description for AudioScholar: Transforming Audio into Actionable Insights for Learners," Version 1.7, Cebu City, Philippines, April 30, 2025.

[4] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, pp. 1–40, Oct. 1998, doi: https://doi.org/10.1109/IEEESTD.1998.88286.

[5] "Gemini API Docs and Reference | Google AI for Developers," Google for Developers. [Online]. Available: https://ai.google.dev/gemini-api/docs.

[6] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, datatracker.ietf.org, Oct. 2012. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6749.

[7] "Authorizing OAuth apps," GitHub Docs. [Online]. Available: https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/authorizing-oauth-apps.

[8] Google. "Firebase." Firebase - Build and Run Successful Mobile and Web Apps. [Online]. Available: https://firebase.google.com/. [Accessed: Mar. 22, 2025].

[9] S. J. Russell and P. Norvig, *Artificial Intelligence : a Modern Approach*, 4th ed. London: Pearson, 2021.

[10] Google. "YouTube Data API." Google Developers. [Online]. Available: https://developers.google.com/youtube/v3. [Accessed: Mar. 22, 2025].

[11] Rescorla, E. (2000). "HTTP Over TLS." RFC 2818, Internet Engineering Task Force (IETF). [Online]. Available: https://datatracker.ietf.org/doc/html/rfc2818. [Accessed: March 10, 2025].

[12] JetBrains. "Kotlin Programming Language." [Online]. Available: https://kotlinlang.org/. [Accessed: March 10, 2025].

[13] D. Jurafsky, J. H. Martin, A. Kehler, Keith Vander Linden, and N. Ward, *Speech and language processing : an introduction to natural language processing, computational linguistics, and speech recognition*. Delhi: Pearson Education, 2008.

[14] Jones, M., Bradley, D., & Sakimura, N. (2015). "OAuth 2.0 for Native Apps." RFC 8252, Internet Engineering Task Force (IETF). [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8252.

[Accessed: March 10, 2025].
[15] ReactJS, ReactJS Documentation. [Online]. Available: https://react.dev/learn. [Accessed: Mar. 22, 2025]. (Also see: https://react.dev/reference/react)
[16] Spring Boot, Spring Boot Reference Documentation. [Online]. Available: https://docs.spring.io/spring-boot/index.html. [Accessed: Mar. 22, 2025]. (Also see: https://spring.io/projects/spring-boot)
[17] Fette, I., & Melnikov, A. (2011). "The WebSocket Protocol." RFC 6455, Internet Engineering Task Force (IETF). [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6455. [Accessed: March 10, 2025].
[18] T. Gur, T. Dilci, İ. Coskun, and B. Delican, "The impact of note-taking while listening on listening comprehension in a higher education context," *International Journal of Academic Research*, vol. 5, no. 1, pp. 93–97, Jan. 2013, doi: https://doi.org/10.7813/2075-4124.2013/5-1/b.16.
[19] X. Wang, "An analysis of note-taking strategies: The effect of translanguaging on content comprehension and knowledge retention," *Journal of Language Teaching*, vol. 1, no. 3, pp. 1–20, Nov. 2021, doi: https://doi.org/10.54475/jlt.2021.020.
[20] F. Bry and A. Y.-S. Pohl, "Large class teaching with Backstage," *Journal of Applied Research in Higher Education*, vol. 9, no. 1, pp. 105–128, Feb. 2017, doi: https://doi.org/10.1108/jarhe-06-2015-0042.
[21] Artz, B., Johnson, M., Robson, D., & Taengnoi, S. (2020). Taking notes in the digital age: Evidence from classroom random control trials. *The Journal of Economic Education*, 51(2), 103–115. https://doi.org/10.1080/00220485.2020.1731386
[22] M. Seddik et al., "People's Democratic Republic of Algeria Ministry of Higher Education and Scientific Research An Investigation of Lectures Note-Taking Strategies and Difficulties The Case of Third Year Students of English at the University of Mohammed." Accessed: Feb. 19, 2025. [Online]. Available: http://dspace.univ-jijel.dz:8080/xmlui/bitstream/handle/123456789/7504/420.382.pdf?sequence=1
[23] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design patterns: elements of reusable object-oriented software*. Reading, MA, USA: Addison-Wesley, 1994.
[24] R. C. Martin, *Clean architecture: a craftsman's guide to software structure and design*. Upper Saddle River, NJ, USA: Prentice Hall, 2018.
[25] Android Developers, Android Developers Documentation. [Online]. Available: https://developer.android.com/docs. [Accessed: Mar. 22, 2025].
[26] Stanford NLP Group, Stanford CoreNLP. [Online]. Available:

https://stanfordnlp.github.io/CoreNLP/. [Accessed: Mar. 22, 2025].

[27] ClickUp Project Management Software. [Online]. Available: https://clickup.com/ (Assumed tool based on user input).

[28] Nhost Backend Development Platform. [Online]. Available: https://nhost.io/ (Service used for storage).

[29] Render Cloud Application Hosting. [Online]. Available: https://render.com/ (Service used for backend hosting).

[30] Google Files API Documentation. [Online]. Available: (Specific URL depends on the exact API used, e.g., Google Drive API or Cloud Storage API).

# 3. DEFINITIONS

- **AI (Artificial Intelligence):** A broad field of computer science focused on creating intelligent systems. In AudioScholar, AI refers to the use of advanced algorithms and models, specifically the Google Gemini AI API, to automatically process audio recordings, generate summaries, and potentially analyze lecture content for recommendations.

- **API (Application Programming Interface):** A set of defined rules and specifications that software programs follow to communicate. AudioScholar utilizes APIs like Google Gemini AI API, YouTube Data API, Firebase Authentication, Google OAuth 2.0, GitHub OAuth 2.0, and potentially Google Files API.

- **AudioScholar:** The name of the software application being specified/designed. An intelligent, multi-user platform (Android mobile app and web interface) to record lecture audio, leverage AI for summarization and recommendations, aiming to transform audio into actionable insights for learners.

- **Audio Upload (Mobile):** Functionality within the mobile application allowing users to upload pre-recorded audio files for processing.

- **Audio Upload (Web):** Functionality within the web interface allowing users to upload pre-recorded audio files for processing.

- **Cloud Synchronization:** The optional process of transferring data (recordings, summaries) between the mobile device and cloud storage (Nhost Storage, Firebase Firestore) for backup and cross-device accessibility.

- **EdTech (Educational Technology):** The use of technology to support and enhance education. AudioScholar falls into this domain.

- **ERD (Entity Relationship Diagram):** A structural diagram used in database design to illustrate relationships between entities. Used in the SDD to visualize data structures.

- **Firebase:** Google's mobile and web application development platform. AudioScholar uses Firebase Authentication, Firestore (database), and potentially Firebase Cloud Messaging (FCM) for push notifications. Nhost Storage is used for file storage instead of Firebase Storage.

- **Freemium Model:** A business model offering basic services free of charge while charging for advanced features. AudioScholar implements this with tiered access based on user status (unauthenticated, free authenticated, premium).

- **Functional Requirements:** Define specific tasks the system must perform (detailed in SRS Section 3.2).

- **GitHub OAuth 2.0:** An implementation of OAuth 2.0 allowing users to authenticate using their GitHub accounts. Used as a login option in AudioScholar.

- **Google Gemini AI API:** Google's advanced AI API suite used by AudioScholar for audio processing, summarization, and potentially recommendations.

- **Google OAuth 2.0:** An open standard authorization protocol enabling users to grant limited access to their Google accounts. Used as a login option in AudioScholar.
- **HTTP (Hypertext Transfer Protocol):** Foundation of data communication for the World Wide Web.
- **HTTPS (Hypertext Transfer Protocol Secure):** Secure version of HTTP using encryption. Used for all communication between AudioScholar components transmitting sensitive data.
- **IEEE (Institute of Electrical and Electronics Engineers):** Professional organization known for standards. IEEE Std 830-1998 is referenced for SRS structure.
- **iOS:** Apple's mobile operating system. Not supported in the initial version of AudioScholar.
- **ISO (International Organization for Standardization):** International standard-setting body. Referenced in SDD for software engineering standards.
- **JDBC (Java Database Connectivity):** API for Java to access databases. Used in the server-side application.
- **JVM (Java Virtual Machine):** Runtime environment for Java bytecode. Kotlin compiles to JVM bytecode.
- **Kotlin:** Modern, statically-typed language used for developing the AudioScholar Android mobile application.
- **Lecture Recording (Mobile):** Core functionality of the mobile app for robust audio recording in online/offline environments.
- **LMS (Learning Management System):** Software for managing learning processes. AudioScholar is initially standalone, not integrated with LMS.
- **Mobile Application (Android):** The AudioScholar application specifically developed for Android devices using Kotlin.
- **NLP (Natural Language Processing):** Branch of AI dealing with computer-human language interaction. Used in AudioScholar for analyzing lecture content (summaries) for keywords/topics.
- **Non-functional Requirements:** Define quality attributes like performance, security, usability (detailed in SRS Section 3.3).
- **OAuth 2.0 (Open Standard for Authorization):** Authorization framework used by AudioScholar for Google and GitHub login.
- **Personalized Learning Material Recommendations:** Feature offering tailored suggestions (initially YouTube) based on summarized lecture content.
- **PowerPoint Integration:** Enhancement feature allowing optional upload of PPT files to improve AI summarization accuracy and contextuality.

- **PPT (PowerPoint Presentation):** Files created by Microsoft PowerPoint, optionally used for context in AudioScholar.
- **ReactJS:** JavaScript library for building user interfaces. Used to develop the AudioScholar web interface.
- **REST (Representational State Transfer):** Architectural style for networked applications. AudioScholar's server-side components use RESTful APIs.
- **Runnable:** Standard Java interface for tasks executed by a thread. Used in Android app (Module 1) for updating recording timer UI.
- **SDD (Software Design Description):** The document outlining the design framework for AudioScholar.
- **SDK (Software Development Kit):** Set of tools for creating applications. AudioScholar uses Firebase SDK, Google Sign-In SDK.
- **Server-side:** Components and functionalities executed on the server (audio processing, summarization, etc.).
- **Snackbar:** Android UI element for brief messages at the screen bottom. Used for feedback in AudioScholar mobile app.
- **SPP (Software Project Proposal):** The initial document outlining the project concept and objectives.
- **Spring Boot:** Open-source Java framework used for developing the AudioScholar server-side application.
- **SRS (Software Requirements Specification):** The document detailing the requirements for AudioScholar.
- **Toast:** Android UI element for simple feedback in a small popup. Used for short messages in AudioScholar mobile app.
- **UAT (User Acceptance Testing):** Testing phase where end-users validate the software.
- **UI (User Interface):** Means of interaction between user and system. Refers to visual/interactive components of AudioScholar mobile/web apps.
- **User Authentication and Account Management:** System responsible for verifying user identities and managing accounts (supports Google/GitHub OAuth, email/password).
- **UUID (Universally Unique Identifier):** 128-bit number for unique identification. Used for entities in AudioScholar.
- **UX (User Experience):** Overall experience of using a product/service. AudioScholar aims for a positive UX.
- **Web Interface (ReactJS):** The browser-based component of AudioScholar developed using

ReactJS, providing access to recordings, summaries, recommendations, and audio upload.

- **WebSocket:** Communication protocol for full-duplex communication. Considered for future real-time enhancements.
- **XML (Extensible Markup Language):** Markup language used for encoding documents. Used for layout definitions in Android development.
- **YouTube Data API:** API for interacting with YouTube data. Used by AudioScholar to fetch learning material recommendations.
- **Google Files API:** API potentially used for handling large file uploads or URIs.

# 4. PROJECT ORGANIZATION

## *4.1. EXTERNAL STRUCTURE*

The project team operates under the College of Computer Studies, Cebu Institute of Technology University. Key external interfaces include:

- **University/Faculty:** Project Adviser (Jasmine A. Tulin) and Overall Capstone Lead (Ralph P. Laviste).
- **External Services/APIs:** Google Gemini AI API, Firebase Services, Nhost Storage, Google OAuth 2.0 API, GitHub OAuth 2.0 API, YouTube Data API, Google Files API, Render (cloud host).
- **Users:** College and university students (primary target audience and testers).

## *4.2. INTERNAL STRUCTURE*

**Team Leader:** Biacolo, Math Lee L.

**Team Members:**

- Terence, John Duterte
- Orlanes, John Nathan
- Barrientos, Claive Justin
- Alpez, Christian Brent

**Project Adviser:** Jasmine A. Tulin

## *4.3. ROLES AND RESPONSIBILITIES*

- **Team Leader (Math Lee L. Biacolo):** Oversees project execution, manages ClickUp taskboard, facilitates communication, ensures alignment with objectives and timeline, proposes and implements requirement/document changes, monitors free tier usage, leads integration, and addresses major roadblocks.
- **Team Members (Terence, Orlanes, Barrientos, Alpez):** Self-assign tasks/subtasks from ClickUp based on sprint goals; design, develop, test, and document features; participate in daily coding meetings/reviews; resolve technical issues collaboratively or escalate if needed.
- **Project Adviser (Jasmine A. Tulin):** Provides academic guidance, reviews progress and documentation, consults regularly with the team (TULIWED11001200).
- **Overall Capstone Lead (Ralph P. Laviste):** Provides high-level program oversight, approves final documentation and product.

# 5. MANAGERIAL PROCESS PLANS

## *5.1. START-UP PLAN*

### *5.1.1. Estimation plan*

Effort estimation relies on weekly checkpoints where progress on self-assigned tasks from ClickUp is reviewed. The Team Leader assesses feasibility and alignment with the overall 8-week deadline, adjusting tasks as needed for efficiency. If tasks take significantly longer than initially anticipated, alternative solutions are explored rather than getting fixated, ensuring progress continues. Re-estimation occurs implicitly through task adjustment and checkpoint reviews.

### *5.1.2. Staffing plan*

The project team consists of the five members identified in section 4.2. Personnel are assigned for the duration of the 8-week project. Tasks are made available by the Team Leader in ClickUp; members choose tasks/subtasks based on availability and interest, ensuring coverage of all required development areas (mobile, web, backend).

### *5.1.3. Resource acquisition plan*

Primarily utilizes freely available software tools (Visual Studio Code, Android Studio). Access to shared services like Firebase and Google Cloud is managed via email invitations. For services like Nhost where sharing is restricted on free tiers, each member creates an individual account and API keys are used interchangeably. No specific hardware acquisition is planned beyond team members' existing development setups and access to Android devices for testing.

### *5.1.4. Project staff training plan*

No formal training plan is established; team members are expected to utilize existing skills in Kotlin, Java, ReactJS, Spring Boot, and related technologies, or acquire necessary knowledge independently through documentation and online resources as development progresses.

## *5.2. WORK PLAN*

### *5.2.1. Work activities*

Work is broken down into phases corresponding to module development, features within those modules, and specific implementation tasks and subtasks. This structure is maintained within the ClickUp taskboard. A high-level example:

- Phase: Module 1 Implementation
  - Feature: General Setup (Module 1)
    - Task: Set up/Verify Mobile Project Structure
    - Task: Set up/Verify Web Project Structure
  - Feature: Lecture Recording (Mobile)
    - Task: Implement Local Recording Storage
      - Subtask: Get Storage Directory Path
      - Subtask: Ensure Directory Exists
      - Subtask: Implement Filename Generation
      - Subtask: Configure MediaRecorder
      - Subtask: Add Error Handling
    - Task: Implement Recording Controls (Start/Stop/Pause/Resume)
    - Task: Implement Foreground Service & Notification (This structure continues for all modules and features).

### *5.2.2. Schedule allocation*

The project adheres to an 8-week overall timeline ending May 06, 2025, structured around 2-week sprints and weekly checkpoints. Specific task start/end dates and durations are managed dynamically within ClickUp based on task dependencies and team progress. Dependencies are managed by blocking subsequent tasks in ClickUp until prerequisites are marked complete.

### *5.2.3. Resource allocation*

Team members self-allocate to tasks available in ClickUp based on sprint priorities defined during planning/checkpoints and individual capacity/expertise. The Team Leader ensures all critical tasks are picked up.

### *5.2.4. Budget allocation*

Not applicable, as there is no monetary budget currently allocated to specific activities. Resource usage focuses on staying within free tier limits.

## *5.3. CONTROL PLAN*

### *5.3.1. Requirements control plan*

Requirements are baselined in the SRS document. If changes are needed, they are proposed and implemented by the Team Leader, who assesses the impact on schedule and other documents, ensuring consistency across the project artifacts. Changes are tracked via document versioning.

### *5.3.2. Schedule control plan*

Schedule progress is tracked via task completion status in ClickUp and reviewed during weekly checkpoints. Deviations are identified by comparing completed work against the planned sprint goals and overall timeline. Corrective actions primarily involve increased team effort or exploring alternative technical solutions if a task proves infeasible; re-scoping is a last resort due to the fixed deadline.

### *5.3.3. Budget control plan*

Control involves monitoring the usage dashboards of Firebase, Nhost, Render, and Google Cloud services to ensure free tier limits are not exceeded. The Team Leader is responsible for this monitoring. If limits are approached, alternative free services or optimization strategies will be investigated.

### *5.3.4. Quality control plan*

Quality is controlled through adherence to defined standards (visual UI, code organization, no production bugs), informal peer reviews during daily coding meetings, and execution of the V&V plan (unit, integration, system, usability testing).

### *5.3.5. Reporting plan*

Progress is reported informally within the team during daily coding meetings. Formal status updates occur during weekly checkpoints with the Team Leader/potentially Adviser. ClickUp provides real-time visibility into task status. Final reports include the Test Document and Final Paper.

### *5.3.6. Metrics collection plan*

- *Time Reduction:* Measured by timing the duration from audio upload/completion to the display of generated content (target ~40-60s for 30min audio).
- *Comprehension Improvement:* Assessed qualitatively via user feedback/surveys during testing.

- *Summary Quality Score:* Evaluated via manual review by target student users based on relevance and completeness during usability testing.

- *User Satisfaction:* Measured using feedback questionnaires/ratings during usability testing (Target: 4.5/5 stars).

- *System Uptime:* Monitored via Render/Firebase dashboards (Target: 99%).

- *User Adoption:* Tracked via basic analytics if feasible within free tiers.

### *5.3.7 Risk management plan*

- *Risk Identification:* Key risks include internet connectivity dependency, external API failure/changes (esp. Gemini), cloud service limitations (upload size, memory), and team member availability.

- *Risk Analysis/Prioritization:* API failures and connectivity issues are high impact. Service limitations have proven significant (requiring workarounds). Team availability is managed through commitment.

- *Risk Mitigation:* Implemented technical workarounds (chunked uploads, Google Files API). Daily testing helps catch issues early. Agile approach allows adaptation. Team commitment addresses time constraints. Monitoring free tier usage prevents service interruption.

- *Risk Monitoring:* Risks are implicitly monitored through daily development activities, testing, and service usage checks.

### *5.3.8 Project closeout plan*

Closeout involves completing all software deliverables, finalizing and submitting all required documentation (Test Document, Final Paper following CIT-U templates), performing a final presentation/demo, and committing the final source code. While a formal retrospective isn't planned, lessons learned are discussed informally. The project might be continued by the team for future development.

# 6. TECHNICAL PROCESS PLANS

## *6.1 PROCESS MODEL*

The project follows the Agile development methodology, specifically Scrum. The lifecycle involves iterative development within 2-week sprints. Key activities include sprint planning (setting goals based on backlog), daily coding meetings (serving as stand-ups), development work, continuous testing, weekly checkpoints (serving as sprint reviews), and ongoing backlog refinement. This model provides flexibility to adapt within the 8-week timeframe.

## *6.2 METHODS, TOOLS, AND TECHNIQUES*

- **Methods/Techniques:** Agile/Scrum, Object-Oriented Design, RESTful API design, NLP techniques (keyword extraction, topic identification), Continuous Testing, Version Control (Git), Branching Strategy (GitHub Flow).

- **Tools/Technologies:**
    - *Project Management:* ClickUp.
    - *Version Control:* Git, GitHub.
    - *IDEs:* Android Studio, Visual Studio Code.
    - *Languages:* Kotlin (Android), Java (Backend), JavaScript (Web).
    - *Frameworks/Libraries:* Android SDK, Spring Boot, ReactJS, Tailwind CSS.
    - *AI/APIs:* Google Gemini AI API, Google Files API, YouTube Data API, Firebase Auth API, Google OAuth 2.0 API, GitHub OAuth 2.0 API, FCM API.
    - *Backend/Cloud:* Firebase (Authentication, Firestore), Nhost Storage, Render (Hosting).
    - *Development Support:* ngrok.

## *6.3 INFRASTRUCTURE PLAN*

- **Development Environment:** Team members use their own workstations with necessary IDEs installed. Testing is performed on available Android devices (targeting Minimum OS: Android 7.0 Nougat). Internet access is required. ngrok is used for local backend testing.

- **Testing Environment:** Testing occurs on development branches using team members' devices and cloud service development instances (Firebase dev project, etc.). Builds are shared and tested via GitHub branches.

- **Production Environment:** Firebase production project, Nhost Storage, Render for backend hosting.

- **Backup/Recovery:** No formal data backup and recovery plan is currently implemented beyond the standard provisions of the cloud service providers (Firebase, Nhost, Render).

## *6.4 PRODUCT ACCEPTANCE PLAN*

- **Criteria:** Acceptance is based on:
    - Demonstration that all functional requirements outlined in the SRS are met.
    - Meeting key non-functional requirements (usability, reliability, performance).
    - Successful generation of summaries within approximately 40-60 seconds for a 30-minute audio file.
    - Positive feedback and validation from usability testing with target student users.
    - Final approval from the Overall Capstone Lead.
- **Process:** Acceptance testing will involve final demonstrations and reviews of the software and documentation by the Project Adviser and Overall Capstone Lead. User feedback from UAT/usability testing will also be considered. Formal acceptance is signified by the Capstone Lead's approval.

# 7. SUPPORTING PROCESS PLANS

## *7.1. CONFIGURATION MANAGEMENT PLAN*

- **Identification:** Project artifacts (code, documents) are identified and managed.
- **Version Control:** Source code is managed using Git/GitHub. Documents are versioned manually.
- **Branching:** GitHub Flow strategy is used (main branch, feature branches).
- **Build Management:** Builds for testing are created directly from feature branches by team members.
- **Change Control:** Managed by the Team Leader as described in 5.3.1.
- **Auditing:** Informal auditing occurs through code reviews and testing.

## *7.2. VERIFICATION AND VALIDATION PLAN*

- **Verification:** Includes informal code reviews during daily meetings and adherence to quality standards (code org, UI). Unit and integration testing are performed by developers.
- **Validation:** Includes system testing, usability testing with target users, UAT (pilot testing), and performance testing to ensure the software meets user needs and requirements.
- **Defect Tracking:** Defects found during testing are communicated directly within the team and resolved; no formal bug tracking tool is used.

## *7.3. DOCUMENTATION PLAN*

- **Documents:** SPP, SRS, SDD, SPMP, Test Document, Final Paper (ACM Format).
- **Standards:** CIT-U templates used for Test Document and Final Paper; ACM format for Final Paper.
- **Responsibilities:** Team members contribute based on module assignments; Team Leader ensures consistency and finalizes documents.
- **Review/Approval:** Documentation reviewed by Project Adviser, approved by Overall Capstone Lead.

## *7.4. QUALITY ASSURANCE PLAN*

QA activities are integrated into the development process: adherence to Agile/Scrum, execution of the V&V plan, informal code reviews, focus on defined quality standards (UI, code org, no prod bugs), and feedback loops through checkpoints and user testing.

## *7.5. REVIEWS AND AUDITS*

- **Management Reviews:** Weekly checkpoints serve as progress reviews. Final review by Capstone Lead for acceptance.
- **Technical Reviews:** Informal code reviews occur during daily coding meetings. Usability testing provides external review of the UI/UX.
- **Audits:** No formal audits are planned.

## *7.6. PROBLEM RESOLUTION PLAN*

Bugs identified during testing are reported directly to the relevant team member(s) and prioritized for fixing. Technical roadblocks unresolved by an individual are brought to the daily coding meeting for collaborative problem-solving or task reassignment.

## *7.7. SUBCONTRACTOR MANAGEMENT PLAN*

Not applicable.

## *7.8. PROCESS IMPROVEMENT PLAN*

No additional plans are defined for this project.

# 8. ADDITIONAL PLANS

No annexes are included.

# 9. PLAN ANNEXES

An index is not included in this document.

# 10. INDEX

An index is not included in this document.